

Hindi Vidya Prachar Samiti's
RAMNIRANJAN JHUNJHUNWALA COLLEGE OF
ARTS , SCIENCE AND COMMERCE
(EMPOWERED AUTONOMOUS)

Optimization Technique



Name : Sudhanshu Bisoi

Roll no : 10401

Class : Msc. Data Science and Artificial Intelligence Part II



Ramniranjan Jhunjhunwala College of Arts, Science and Commerce

Department of Data Science and Artificial Intelligence

CERTIFICATE

This is to certify Sudhanshu Bisoi of Msc. Data Science and Artificial Intelligence Roll No-10401 has successfully completed the practical of Optimization Technique during the Academic Year 2025-2026.

Date :

**(Prof. Rahul Yadav)
Prof-In-Charge**

External Examiner

INDEX

Sr. No.	Practical	Date	Sign
1	Classical Optimization Technique	22/08/25	
2	Unconstrained optimization : Elimination Method	22/08/25	
3	Unconstrained optimization : Interpolation Method	22/08/25	
4	Unconstrained optimization : Direct Root Method	22/08/25	
5	Unconstrained optimization : Equality Constraints	22/08/25	
6	Unconstrained optimization : Inequality Constraints	22/08/25	

Practical No.1 - Classical Optimization Technique

1. Golden Search Method -

```
In [1]: # tol = 1e-5 stands for tolerance, which is a stopping criteria for the algorithm.
import math
def golden(f,a,b, tol=1e-2):
    phi = (1+5**0.5)/2
    while b-a > tol:
        c = b - (b-a)/phi
        d = a + (b-a)/phi
        if f(c) < f(d): b=d
        else: a = c
    return (a+b)/2
f = lambda x: (x-3)**2 + 2
print(golden(f,0,6))

3.0013593115613544
```

2. Steepest Descent on quadratic method -

```
In [2]: import numpy as np
A = np.diag([2.0,8.0])
B = np.array([2.0,16.0]) # since grad = Ax-b; here f = (x-1)^2+4(y-2)^2
x = np.array([0.0,0.0])
for k in range(5):
    g = A@x - B
    alpha = (g@g) / (g@(A@g))
    x = x - alpha*g
    print(k,x)

0 [0.25291829 2.0233463 ]
1 [0.96704051 1.93408103]
2 [0.97537657 2.00076948]
3 [0.99891367 1.99782734]
4 [0.99918842 2.00002536]
```

3. Newton 1-D Method -

```
In [3]: def newton_minimize(f1, f2, x0, iters=5):
    x = x0
    for _ in range(iters):
        x = x-f1(x)/f2(x)
    return x
f1 = lambda x: 4*x**3 - 9*x**2
f2 = lambda x: 12*x**2 - 18*x
print(newton_minimize(f1,f2,3.0))

2.2500000000031966
```

Practical No.2 - Unconstrained optimization : Elimination Method

1. Interval halving Method -

```
In [4]: import math
# Define objective function
f = lambda x: (x - 2.5)**2 + 3    #Minimum at x=2.5 ,f(x)=3

def interval_halving(f,a,b, tol=1e-3):
    k = 0
    while (b-a) > tol:
        k += 1
        mid = (a + b) / 2
        l = (a + mid) / 2
        r = (mid + b) / 2

        f_l, f_mid, f_r = f(l), f(mid), f(r)

        #Elimination logic
        if f_l < f_mid:
            b = mid
        elif f_r < f_mid:
            a = mid
        else:
            a,b = l,r
        print(f"Iter {k}: Interval=({a:.4f}, {b:.4f})")
    return (a + b) / 2

x_opt = interval_halving(f,0,6)
print("Interval Halving Optimum:", x_opt, "f(x)=", f(x_opt))
```

```
Iter 1: Interval=(1.5000, 4.5000)
Iter 2: Interval=(1.5000, 3.0000)
Iter 3: Interval=(2.2500, 3.0000)
Iter 4: Interval=(2.2500, 2.6250)
Iter 5: Interval=(2.4375, 2.6250)
Iter 6: Interval=(2.4375, 2.5312)
Iter 7: Interval=(2.4844, 2.5312)
Iter 8: Interval=(2.4844, 2.5078)
Iter 9: Interval=(2.4961, 2.5078)
Iter 10: Interval=(2.4961, 2.5020)
Iter 11: Interval=(2.4990, 2.5020)
Iter 12: Interval=(2.4990, 2.5005)
Iter 13: Interval=(2.4998, 2.5005)
Interval Halving Optimum: 2.5001220703125 f(x)= 3.000000014901161
```

2. Fibonacci Search Method -

```
In [5]: def f(x):
        return (x - 2)**2 # Example function to minimize

def fibonacci(n):
    fib = [1, 1]
    for i in range(2, n):
        fib.append(fib[-1] + fib[-2])
    return fib

def fibonacci_search(f, a, b, n=10):
    fib = fibonacci(n+1)
    k = 0
    l = b - a

    # Initial test points
    x1 = a + (fib[n-2]/fib[n]) * l
    x2 = a + (fib[n-1]/fib[n]) * l
    f1, f2 = f(x1), f(x2)

    for i in range(1, n-1):
        k += 1
        if f1 > f2:
            a = x1
            x1 = x2
            f1 = f2
            x2 = a + (fib[n-i-1]/fib[n-i]) * (b - a)
            f2 = f(x2)
        else:
            b = x2
            x2 = x1
            f2 = f1
            x1 = a + (fib[n-i-2]/fib[n-i]) * (b - a)
            f1 = f(x1)
        print(f"Iter {k}: Interval = ({a:.4f}, {b:.4f})")

    return (x1 + x2) / 2

# Run the optimization
x_opt = fibonacci_search(f, 0, 6, n=12)
print("Fibonacci Search Optimum:", x_opt, "f(x) =", f(x_opt))
```

```
Iter 1: Interval = (0.0000, 3.7082)
Iter 2: Interval = (1.4163, 3.7082)
Iter 3: Interval = (1.4163, 2.8326)
Iter 4: Interval = (1.4163, 2.2918)
Iter 5: Interval = (1.7511, 2.2918)
Iter 6: Interval = (1.7511, 2.0858)
Iter 7: Interval = (1.8798, 2.0858)
Iter 8: Interval = (1.9571, 2.0858)
Iter 9: Interval = (1.9571, 2.0343)
Iter 10: Interval = (1.9828, 2.0343)
Fibonacci Search Optimum: 2.008583690987124 f(x) = 7.36797509624371e-05
```

Practical No. 3 - Unconstrained optimization : Interpolation Method

Quadratic Interpolation Method -

```
In [6]: #Quadratic Interpolation Method
import numpy as np

# Objective function
f = lambda x: (x - 2.5)**2 + 3 # Minimum at x=2.5, f(x)=3

def quadratic_interpolation(f, x0, x1, x2, tol=1e-5, max_iter=20):
    for k in range(max_iter):
        f0, f1, f2 = f(x0), f(x1), f(x2)

        # Formula for vertex of parabola
        numerator = ((x1**2 - x2**2)*f0 + (x2**2 - x0**2)*f1 + (x0**2 - x1**2)*f2)
        denominator = ((x1 - x2)*f0 + (x2 - x0)*f1 + (x0 - x1)*f2)

        if denominator == 0:
            break

        x_min = 0.5 * numerator / denominator
        f_min = f(x_min)

        print(f"Iter {k+1}: x_min={x_min:.6f}, f(x)={f_min:.6f}")

        # Replace the point with highest function value
        if f0 > f1 and f0 > f2:
            x0 = x_min
        elif f1 > f0 and f1 > f2:
            x1 = x_min
        else:
            x2 = x_min

        # Stopping criterion
        if max(abs(x_min - x0), abs(x_min - x1), abs(x_min - x2)) < tol:
            return x_min, f_min

    return x_min, f_min

# Run method
x_opt, f_opt = quadratic_interpolation(f, 0, 3, 5)
print("\nQuadratic Interpolation Optimum: x =", round(x_opt, 5), "f(x)=", round(f_opt, 5))
```

Iter 1: x_min=2.500000, f(x)=3.000000

Iter 2: x_min=2.500000, f(x)=3.000000

Quadratic Interpolation Optimum: x = 2.5 f(x)= 3.0

Practical No. 4 - Unconstrained optimization : Direct Root Method

1. Newton-Raphson Method -

```
In [7]: #Newton-Raphson Method
#Function and derivatives
f = lambda x: x**4 - 3*x**3 + 2
f1 = lambda x: 4*x**3 - 9*x**2 #First Derivative
f2 = lambda x: 12*x**2 - 18*x #Second derivative

def newton_root(f1,f2,x0, tol=1e-6, max_iter=20):
    x = x0
    for k in range(max_iter):
        x_new = x - f1(x)/f2(x)
        print(f"Iter {k+1}: x={x_new:.6f}, f(x)={f(x_new):.6f}")
        if abs(x_new - x) < tol:
            return x_new
        x = x_new
    return x

x_opt = newton_root(f1,f2,3.0)
print("\nNewton-Raphson Optimum: x =", round(x_opt, 5), "f(x)=", round(f(x_opt), 5))
```

```
Iter 1: x=2.500000, f(x)=-5.812500
Iter 2: x=2.291667, f(x)=-6.524954
Iter 3: x=2.251462, f(x)=-6.542947
Iter 4: x=2.250002, f(x)=-6.542969
Iter 5: x=2.250000, f(x)=-6.542969
Iter 6: x=2.250000, f(x)=-6.542969
```

Newton-Raphson Optimum: x = 2.25 f(x)= -6.54297

2. Secant Method -

```
In [8]: #Secant Method
def secant_root(f1,x0,x1,tol=1e-6, max_iter=20):
    for k in range(max_iter):
        f0, f1_val = f1(x0), f1(x1)
        if abs(f1_val - f0) < 1e-12: break
        x2 = x1 - f1_val*(x1-x0)/(f1_val-f0)
        print(f"Iter {k+1}: x={x2:.6f}, f(x)={f(x2):.6f}")
        if abs(x2 - x1) < tol:
            return x2
        x0, x1 = x1, x2
    return x1

x_opt = secant_root(f1,2.0,3.0)
print("\nSecant Method Optimum: x =", round(x_opt, 5), "f(x)=", round(f(x_opt), 5))
```

```
Iter 1: x=2.129032, f(x)=-6.405214
Iter 2: x=2.194468, f(x)=-6.512763
Iter 3: x=2.256766, f(x)=-6.542503
Iter 4: x=2.249655, f(x)=-6.542968
Iter 5: x=2.249998, f(x)=-6.542969
Iter 6: x=2.250000, f(x)=-6.542969
Iter 7: x=2.250000, f(x)=-6.542969
```

Secant Method Optimum: x = 2.25 f(x)= -6.54297

3. Bisection Method

```
In [9]: #Bisection Method
def bisection_root(f1,a,b,tol=1e-6, max_iter=50):
    for k in range(max_iter):
        mid = (a+b)/2
        if f1(a)*f1(mid) < 0:
            b = mid
        else:
            a = mid
        print(f"Iter {k+1}: x={mid:.6f}, f(x)={f1(mid):.6f}")
        if abs(b - a) < tol:
            return mid
    return (a + b)/2

x_opt = bisection_root(f1,2,3)
print("\nBisection Method Optimum: x =", round(x_opt, 5), "f(x)=", round(f1(x_opt), 5))
```

```
Iter 1: x=2.500000, f(x)=-5.812500
Iter 2: x=2.250000, f(x)=-6.542969
Iter 3: x=2.375000, f(x)=-6.372803
Iter 4: x=2.437500, f(x)=-6.146225
Iter 5: x=2.468750, f(x)=-5.993377
Iter 6: x=2.484375, f(x)=-5.906521
Iter 7: x=2.492188, f(x)=-5.860416
Iter 8: x=2.496094, f(x)=-5.836686
Iter 9: x=2.498047, f(x)=-5.824650
Iter 10: x=2.499023, f(x)=-5.818589
Iter 11: x=2.499512, f(x)=-5.815548
Iter 12: x=2.499756, f(x)=-5.814025
Iter 13: x=2.499878, f(x)=-5.813263
Iter 14: x=2.499939, f(x)=-5.812881
Iter 15: x=2.499969, f(x)=-5.812691
Iter 16: x=2.499985, f(x)=-5.812595
Iter 17: x=2.499992, f(x)=-5.812548
Iter 18: x=2.499996, f(x)=-5.812524
Iter 19: x=2.499998, f(x)=-5.812512
Iter 20: x=2.499999, f(x)=-5.812506
```

```
Bisection Method Optimum: x = 2.5 f(x)= -5.81251
```

Practical No. 5 - Unconstrained optimization : Equality Constraints

1. Minimize $f(x) = x^2 + y^2$ subject to $x+2y=6$

```
In [10]: import sympy as sp

#Variable
x,y, lam= sp.symbols('x y lam')

#Objective function
f = x**2 + y**2

#Constraint
h = x + 2*y- 6

#Lagrangian
L = f + lam*h

#first order condition
eqs = [ sp.diff(L,v) for v in [x,y]] + [h]

#Solve system of equation
sol = sp.solve(eqs, (x,y,lam), dict=True)
print("Solution:",sol)

#Extract first solution dictionary
sol_dict = sol[0]

#Evaluate f(x,y) at solution
opt_val = f.subs(sol_dict)
print("Optimal value f(x,y)=",opt_val)

Solution: [{x: 6/5, y: 12/5, lam: -12/5}]
Optimal value f(x,y)= 36/5
```

2. Minimize $f(x) = x^2 + y^2$ subject to $x=y=4, x-y=2$

```
In [11]: import sympy as sp

#Variable
x,y, lam1, lam2 = sp.symbols('x y lam1 lam2')

#Objective function
f = x**2 + y**2

#Constraint
h1 = x + y - 4
h2 = x - y - 2

#Lagrangian
L = f + lam1*h1 + lam2*h2

#first order condition
eqs = [ sp.diff(L,v) for v in [x,y]] + [h1,h2]

#Solve system of equation
sol = sp.solve(eqs, (x,y,lam1,lam2), dict=True)
print("Solution:",sol)

#Extract first solution dictionary
sol_dict = sol[0]

#Evaluate f(x,y) at solution
opt_val = f.subs(sol_dict)
print("Optimal value f(x,y)=",opt_val)

Solution: [{x: 3, y: 1, lam1: -4, lam2: -2}]
Optimal value f(x,y)= 10
```

Practical No. 6 - Unconstrained optimization : Inequality Constraints

```
In [12]: from scipy.optimize import minimize

#Define the objective function
def objective(x):
    return x[0]**2 + x[1]**2

#Define the inequality constraint (g(x) <= 0)
def constraint1(x):
    return x[0] + x[1] - 1 #x0 + x1 <= 1 => x0 + x1 - 1 <= 0

#Define the bounds for the variables (if any)
bounds = ((0, None), (0, None)) #x0 >= 0, x1 >= 0

#Define the constraints dictionary
constraints = ({'type':'ineq','fun':constraint1})

#Initial guess
x0 = [0.5,0.5]

#Solve the optimization problem
result = minimize(objective,x0,method='SLSQP',bounds=bounds,constraints=constraints)

print("optimal x:", result.x)
print("optimal value:", result.fun)
print("success:", result.success)
print("message:", result.message)

optimal x: [0.5 0.5]
optimal value: 0.5
success: True
message: Optimization terminated successfully
```